

# Cosmetique AI — Guide de test (tout-en-un)

Lis seulement ce fichier. Fais les étapes dans l'ordre.

## 1. C'est quoi ?

Un studio web (en français) qui crée une campagne Instagram / Facebook / LinkedIn à partir d'une photo produit :

- le produit photographié reste intact ;
- seul le décor (arrière-plan) est généré ;
- le texte marketing ne peut pas inventer de claims.

Schéma simple :

```
Ton navigateur → site Docker (localhost)
                  ↓
                Colab GPU T4 (Google)
                  ↓
                ZIP campagne (8 fichiers)
```

## 2. Ce dont tu as besoin

- Un compte Google (pour Colab)
- Docker Desktop installé et démarré
- Accès au repo GitHub (Jesse t'ajoute comme collaboratrice)
- Accès complet au dossier Google Drive ci-dessous (Jesse t'a déjà invitée en accès complet)
- Une photo produit autorisée pour le test

### Liens

· Quoi · Lien ·  
· ----- ·

· GitHub (code) · <https://github.com/jessem8/cosmetique-ai> — branche `codex/implementation` ·  
· Google Drive (même projet, accès complet déjà partagé avec toi) ·  
<https://drive.google.com/drive/folders/1ADA3bmL8NTdYto1lheDGE16dRkC53ldM> ·  
· Notebook Colab · dans le projet : `notebook/pipeline\_ia\_cosmetique.ipynb` ·

Le dossier Drive doit contenir `Stage\_1\_ouvrier` avec `ai\_core` dedans (c'est normal). Ouvre-le avec le compte Google qui a reçu l'invitation.

---

## 3. Récupérer le code (choisis une option)

### Option A — GitHub (recommandé)

1. Accepte l'invitation GitHub de Jesse.
2. Clone le repo, branche `codex/implementation` :

```
git clone -b codex/implementation https://github.com/jessem8/cosmetique-ai.git
cd cosmetique-ai
```

### Option B — Drive

1. Accepte l'invitation Drive (accès complet) si tu ne l'as pas encore fait.

2. Ouvre le lien Drive ci-dessus avec ton compte Google.
3. Télécharge le dossier `Stage\_1\_ouvrier` (ou travaille depuis une copie locale).
4. Ouvre ce dossier dans PowerShell (`cd` vers ce dossier).

Tu travailles toujours à la racine du projet (là où se trouvent `docker/`, `notebook/`, `frontend/`).

---

## 4. Colab (GPU) — à faire en premier

Sans Colab prêt, le site s'ouvre mais les générations échouent. C'est voulu.

1. Va sur <https://colab.research.google.com/>
2. Fichier → Importer un notebook  
choisis `notebook/pipeline\_ia\_cosmetique.ipynb` (depuis le clone GitHub ou le Drive).
3. Exécution → Modifier le type d'exécution
  - Accélérateur : GPU
  - Type GPU : T4
- Enregistrer
4. Exécution → Tout exécuter
5. Autorise l'accès à Google Drive quand Colab le demande.
6. Attends la fin (la première fois peut être longue : téléchargement des modèles).

Quand c'est prêt, le notebook affiche une fois :

- une URL `https://...trycloudflare.com`
- un token (long mot de passe)

Garde-les sous la main. Ne les mets pas dans Git ni dans une capture publique.

Si le notebook dit qu'il ne trouve pas `Stage\_1\_ouvrier/ai\_core` : vérifie que le dossier Drive ouvert est bien celui du lien ci-dessus (une seule copie).

---

## 5. Docker (site local)

1. Démarre Docker Desktop et attends qu'il soit prêt.
2. À la racine du projet :

```
Copy-Item .\docker\.env.example .\docker\.env
```

3. Ouvre `docker/.env` dans un éditeur et :
  - remplace tous les `replace\_...` par des mots de passe / secrets longs (tu peux générer avec :  
``python -c "import secrets; print(secrets.token_urlsafe(48))"``);
  - mets :

```
AI_SERVICE_URL=https://XXXX.trycloudflare.com
AI_SERVICE_TOKEN=le_token_affiché_par_colab
```

4. Lance :

```
docker compose --env-file .\docker\.env -f .\docker\docker-compose.yml up --build -d
```

5. Ouvre <http://localhost> dans le navigateur.

Si Colab était déjà lancé et que tu changes l'URL/token plus tard :

```
docker compose --env-file .\docker\.env -f .\docker\docker-compose.yml up -d --force-recreate
backend worker
```

---

## 6. Tester le produit (ce que tu observes)

1. Sur `http://localhost` → Créer un compte → se connecter.
2. Uploader une photo produit autorisée.
3. Remplir le brief (FR ou EN) → lancer une génération.
4. Observer les vraies étapes (analyse, extraction, fond, composition, etc.) — pas de fausse barre de % inventée.
5. Quand c'est terminé, télécharger le ZIP et vérifier qu'il contient exactement :

```
instagram.jpg
facebook.jpg
linkedin.jpg
copy.json
cutout.png
mask.png
background.jpg
manifest.json
```

Si Colab se coupe en cours de route, le site doit afficher une erreur claire (`AI_RUNTIME_LOST` / service indisponible) — pas un faux succès. Dans ce cas : relancer Colab (Tout exécuter), recopier URL + token dans `.env`, recréer `backend` + `worker` (commande ci-dessus).

---

## 7. Ce qui est déjà vérifié sans Colab

Jesse a déjà fait tourner les tests locaux (preuves dans `docs/handoff/verification-evidence.md`) :

- tests AI / backend / frontend + E2E navigateur ;
- Docker qui démarre (site + base + worker) ;
- modèles Hugging Face pinés.

Ce que tu valides aujourd'hui : le chaînon manquant = Colab T4 + une vraie génération de bout en bout.

---

## 8. En cas de blocage (rapide)

· Problème · Que faire ·

· ----- ·

- Pas de GPU T4 dans Colab · Réessayer, ou Colab Pro si le free ne donne pas de T4 ·
- Notebook ne trouve pas `ai_core` · Utiliser le Drive du lien ci-dessus ; une seule `Stage_1_ouvrier` ·
- Site ne s'ouvre pas · Docker Desktop démarré ? `docker compose ... up -d` relancé ? ·
- Génération échoue tout de suite · URL/token Colab dans `.env` ? Backend/worker recréés ? Colab encore connecté ? ·
- Tunnel mort · Colab redémarré → nouvelle URL + token → maj `.env` → recréer backend/worker ·

---

C'est tout. Ordre à retenir : Colab T4 → URL+token → Docker → localhost → générer → ZIP.